

P0009r2 : Polymorphic Multidimensional Array Reference

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0009r2
Date: 2016-05-27
Reply-to: hcedwar@sandia.gov, balelbach@lbl.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Bryce Lelbach
Contact: balelbach@lbl.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Robin Maffeo
Contact: Robin.Maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com
Audience: Library Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0009.rst

Revision History	
P0009r0	Original multidimensional array reference paper with motivation, specification, and examples.
P0009r1	Revised with renaming from view to array_ref and allow unbounded rank through variadic arguments.
P0009r2 (<i>current</i>)	Adding details for layout mapping. Move motivation, examples, and relaxed array declaration to separate papers.
References	
P0331	Multidimensional array reference motivation and examples
P0332	Relaxed array declaration
P0122	span: bounds-safe views for sequences of objects
earlier related papers: N4512, N4355, N4300, N4222	

1 Array Reference

The header `<array_ref>` defines types and functions for a multidimensional array reference into a contiguous span of objects.

1.1 Header `<array_ref>` synopsis

```
namespace std {
namespace experimental {

    template< typename DataType , typename ... Properties >
    struct array_ref ;

    template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
    array_ref< /* deduced */ >
    subarray( const array_ref< DataType, Properties ... > & , SliceSpecifiers ... ) noexcept;

}}

namespace std {
namespace experimental {
namespace array_property {

    template< typename >
    struct is_array_property /* = std::integral_constant<bool,?> */ ;

}
```

```

template< typename T >
using is_array_property_v = is_array_property<T>::value ;

// array dimension property
template < typename IntegralType , IntegralType ... Dims >
struct dimension_typed ;

template< size_t ... Dims >
using dimension = dimension_typed< size_t , Dims ... > ;

// array layout property
struct layout_right ;
struct layout_left ;
struct layout_stride ;

template <typename T> struct is_layout ;
template <typename T> constexpr bool is_layout_v = is_layout<T>::value;

// bounds checking property
template< bool Enable >
struct bounds_check_if ;

using bounds_check = bounds_check_if< true > ;

// Assignability property
template< class array_ref_V , class array_ref_U >
struct is_assignable /* : std::integral_constant<bool,?> {} */ ;

template < class array_ref_V , class array_ref_U >
using is_assignable_v = is_assignable< array_ref_V , array_ref_U >::value ;

// subarray support
struct all_type {};
constexpr all_type all = all_type();
}}}

```

1.2 Layout Mapping

The multidimensional array reference `array_ref` maps multi-indices to objects by composing a *layout mapping* with a sequence of objects. The *domain* of a layout mapping is multidimensional index space defined by the Cartesian product of integral extents, $[0..extent<0>)\times[0..extent<1>)\times\dots$. The *codomain* of a layout mapping is a subset of span of integer values.

1.2.1 Multidimensional Index Space

```

namespace std {
namespace experimental {
namespace array_property {

template < typename IntegralType , IntegralType ... Dims>
struct dimension_typed {
    // types

    using size_type = /* implementation-defined */ ;
    using value_type = IntegralType ;

    // constructors, copy, assignment, destructor

    constexpr dimension_typed() noexcept ;

    template <typename... IntegralArgs>
    constexpr dimension_typed(IntegralArgs... dynamic_dims) noexcept;

    constexpr dimension_typed(dimension_typed && rhs) noexcept ;
    constexpr dimension_typed(dimension_typed const& rhs) noexcept ;
    dimension_typed & operator=(dimension_typed && rhs) noexcept ;
    dimension_typed & operator=(const dimension_typed & rhs) noexcept ;

    ~dimension();

    // observers

    static constexpr size_type rank() noexcept;
    static constexpr size_type rank_dynamic() noexcept;

    template< size_type ith >

```

```

static constexpr value_type static_extent() noexcept ;

constexpr value_type extent( size_type ith ) const noexcept ;

constexpr value_type size() const noexcept ;
};

}}}

```

```

template < typename IntegralType , IntegralType ... Dims>
struct dimension_typed

```

Requires: `is_integral_v<IntegralType>` and `( 0 <= Dims )...`

Effects: Each non-zero `Dims` denotes a static extent. Each zero `Dims` denotes a dynamic extent.. `rank() == sizeof...(IntegralType).rank_dynamic()` is the number of zero valued members of `Dims...`

```
constexpr dimension_typed() noexcept
```

Effects: Constructs a `dimension_typed` object such that the value of the `ith` member of `Dims`, denoted by `Dims[ith]`, is `Dims[ith] == operator()[ith]`.

```

template< typename ... IntegralArgs >
constexpr dimension_typed( IntegralArgs ... dynamic_dim ) noexcept

```

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value`. Each `dynamic_dims` is non-negative. `sizeof...(IntegralArgs) == rank_dynamic()`.

Effects: Constructs a `dimension_typed` object such that the value of each dynamic extent is initialized by the corresponding value from `dynamic_dims...`

```

template< size_type ith >
static constexpr value_type static_extent() noexcept ;

```

Requires: `0 <= ith`.

Returns: When `ith < sizeof...(Dims)` value of the *ith* member of `Dims...`, otherwise 1.

```
constexpr value_type extent(size_type ith) const noexcept
```

Requires: `0 <= ith`.

Returns: When `ith < rank()` the *ith* extent, otherwise 1.

```
constexpr value_type size() const noexcept
```

Returns: Product of the extents.

1.2.2 Layout Mapping Concept

```

struct /* layout_concept */ {

    template< typename DimensionType >
    struct mapping : DimensionType {

        // types and constants

        static constexpr bool is_always_unique      = /* layout specific */ ;
        static constexpr bool is_always_contiguous = /* layout specific */ ;
        static constexpr bool is_always_regular    = /* layout specific */ ;

        // constructors, copy, assignment, destructor

        constexpr mapping() noexcept ;
        constexpr mapping( mapping && ) noexcept ;
        constexpr mapping( mapping const & ) noexcept ;
        mapping & operator = ( mapping && ) noexcept ;
        mapping & operator = ( mapping const & ) noexcept ;

        constexpr explicit mapping( layout_concept const & );

        template< typename ... IntegralArgs >
        constexpr mapping( IntegralArgs ... dynamic_dimensions );

        // observers

```

```
constexpr bool is_unique() const noexcept ;
constexpr bool is_contiguous() const noexcept ;
constexpr bool is_regular() noexcept ;

constexpr value_type span() const noexcept ;
constexpr value_type stride( size_type ith ) const noexcept ;

// mapping operator

template< typename ... IntegralArgs >
constexpr value_type operator()( IntegralArgs ... indices ) const noexcept ;
};
};
```

```
template< typename DimensionType >
struct mapping : DimensionType
```

Requires: DimensionType is dimension_typed.

Effects: The domain of the mapping is defined by a DimensionType object.

```
constexpr value_type mapping::span() const noexcept ;
```

Returns: The upper bound for a span of integral values $[0..span())$ such that the codomain of the mapping is a subset of this span.

```
static constexpr bool mapping::is_always_unique
static constexpr bool mapping::is_always_contiguous
static constexpr bool mapping::is_always_regular
constexpr bool mapping::is_unique() const noexcept ;
constexpr bool mapping::is_contiguous() const noexcept ;
constexpr bool mapping::is_regular() const noexcept ;
```

A layout mapping is *unique* if each multi-index is mapped to a unique value within $[0..span())$.

A layout mapping is *contiguous* if the codomain is equal to $[0..span())$.

A layout mapping that is unique and contiguous is *bijective* and has `size() == span()`.

A *regular* layout mapping M has constant striding between multi-index coordinates. Let `indices_V...` and `indices_U...` be multi-indices in the domain space such that all coordinates are equal except for the *ith* coordinate where `indices_V[ith] = indices_U[ith] + 1`. Then $M.stride(ith) = M(indices_V...) - M(indices_U...)$ is constant for all coordinates.

```
constexpr value_type stride( size_type ith ) const noexcept ;
```

Requires: `is_regular()`.

Returns: When $r < rank()$ the distance between members when the index of coordinate r is incremented by one, otherwise 0.

```
template< typename ... IntegralArgs >
constexpr size_type mapping::operator()( IntegralArgs ... indices ) const noexcept ;
```

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value.rank() <= sizeof...(IntegralArgs)`. The *ith* coordinate of `indices...`, denoted as `indices[ith]`, is valid: $0 \leq indices[ith] < extent(ith)$.

Returns: The value of the mapping from the dimension domain space to the $[0..span())$ range space.

Remark: An implementation may have rank-specific overloads to better enable optimization of the member access operator. Since `extent(ith)==1` for $rank() \leq ith$ then extra zero-value indices are valid.

1.2.3 Standard Layouts

```
array_property::layout_right::mapping
```

The `layout_right::mapping` is always bijective, regular, and its strides monotonically increase from right to left. Note that the stride of the right-most coordinate is one, `stride(rank()-1)==1`.

```
array_property::layout_left::mapping
```

The `layout_left::mapping` is always bijective, regular, and its strides monotonically increase from left to right. Note that the stride of the left-most coordinate is one, `stride(0)==1`.

array_property::layout_stride::mapping

The layout_stride::mapping is always regular.

Remark: This is typically the layout of a subarray obtained from a layout_left or layout_right array.

1.3 class template array_ref

```
namespace std {
namespace experimental {

template <typename DataType, typename... Properties>
struct array_ref {
    // types

    using layout      = /* deduced from Properties... */ ;
    using value_type  = /* deduced from DataType */ ;
    using reference    = /* deduced from DataType and Properties... */ ;
    using pointer      = /* deduced from DataType */ ;
    using size_type    = /* implementation defined */ ;

    using iterator      = /* deduced from DataType */ ;
    using const_iterator = /* deduced from DataType */ ;
    using reverse_iterator = reverse_iterator<iterator> ;
    using const_reverse_iterator = reverse_iterator<const_iterator> ;

    // constructors, copy, assignment, and destructor

    constexpr array_ref() noexcept;
    constexpr array_ref(array_ref&& rhs) noexcept ;
    constexpr array_ref(array_ref const& rhs) noexcept ;
    array_ref& operator=(array_ref&& rhs ) noexcept ;
    array_ref& operator=(array_ref const& rhs ) noexcept ;

    template <typename... IntegralArgs>
    explicit constexpr array_ref(pointer p, IntegralArgs... dynamic_dims) noexcept;

    explicit constexpr array_ref(pointer p, layout const&) noexcept;

    template <typename UType, typename ... UProperties>
    constexpr array_ref(array_ref<UType, UProperties...> const& rhs) noexcept;

    template <typename UType, typename ... UProperties>
    array_ref& operator=(array_ref<UType , UProperties...> const& rhs) noexcept;

    ~array_ref() noexcept ;

    // observers of domain index space

    static constexpr size_type rank() noexcept;
    static constexpr size_type rank_dynamic() noexcept;

    constexpr size_type size() const noexcept;

    constexpr size_type extent(size_type ith) const noexcept;

    // observers of mapping from domain index space to range value space

    static constexpr bool is_always_unique      = /* deduced */ ;
    static constexpr bool is_always_contiguous = /* deduced */ ;
    static constexpr bool is_always_regular    = /* deduced */ ;

    constexpr bool is_unique() const noexcept;
    constexpr bool is_contiguous() const noexcept;
    constexpr bool is_regular() noexcept;

    constexpr size_type stride(size_type rank) const noexcept;

    constexpr size_type span() const noexcept;

    template <typename... IntegralArgs>
    static constexpr size_type required_span( IntegralArgs ... dynamic_dims ) const noexcept;

    static constexpr size_type required_span( layout const & ) const noexcept;

    // element and data access
```

```
constexpr pointer data() const noexcept;

template <typename... IntegralArgs>
reference operator()(IntegralArgs... indices) const noexcept;

reference operator[](size_type idx) const noexcept;

// iterator support, requires is_contiguous

constexpr iterator begin() const noexcept ;
constexpr iterator end()    const noexcept ;
constexpr const_iterator cbegin() const noexcept ;
constexpr const_iterator cend()    const noexcept ;
constexpr reverse_iterator rbegin() const noexcept ;
constexpr reverse_iterator rend()    const noexcept ;
constexpr const_reverse_iterator crbegin() const noexcept ;
constexpr const_reverse_iterator crend()    const noexcept ;
};
}}
```

1.3.1 Template arguments

`array_ref` accepts the `DataType` and `Properties...` template arguments.

`DataType`

`DataType` declares the type of array elements and optionally the dimension of the array, if those dimensions can be expressed as an array type (8.3.4.p3).

Requires: The `DataType` argument is either an array type (8.3.4.p3) or complete object type.

`Properties...`

The `Properties...` argument is a pack of array properties.

Requires: `array_property::is_array_property_v< Properties >` for every member of the pack. If `DataType` is an array type then `Properties...` shall not contain an `array_property::dimension` and the array dimensions are deduced from `DataType`.

1.3.2 Array Dimensions

The dimensions of an instantiated `array_ref` type are defined by either the `DataType` array type (8.3.4.p3) or an `array_property::dimension` type in the `Properties...` pack.

If `DataType` is an array type each declared extent `[N]` denotes a static extent and each omitted extent `[ ]` denotes a dynamic extent. For example, `array_ref<double[ ][3][8]>` declares a rank-three array with dynamic extent in coordinate zero, static extent of 3 in coordinate one, and static extent of 8 in coordinate two.

If an `array_property::dimension` is given then each positive integral value denotes a static extent and each zero value denotes a dynamic extent. For example, `array_ref<double,array_property::dimension<0,3,8> >` declares a rank-three array with dynamic extent in coordinate zero, static extent of 3 in coordinate one, and static extent of 8 in coordinate two.

If `DataType` is a complete object type and no `dimension` is given then the array is rank-zero.

1.3.3 Types

```
using layout = /* deduced from Properties... */ ;
```

Layout property of the array. If `Properties...` does not include a layout property then `layout` is void.

```
using value_type = /* deduced from DataType */ ;
```

Type of the objects referenced by the array.

Remark: A likely implementation is using `value_type = typename std::remove_all_extents< DataType >::type`.

```
using reference = /* deduced from DataType and Properties... */ ;
```

The type returned by the member access operator. Typically this will be `value_type&`. The *constness* of the refenced objects is defined by the *constness* of `value_type`. For example, references to objects of `array_ref<const double[ ][3][8]>`

are const. [Note: The reference type may be a proxy depending upon `Properties...`. For example, if a property indicates that all member references are to be atomic then the reference type would be a proxy conforming to *atomic-view-concept* introduced in paper P0019. - end note]

```
using pointer = /* deduced from DataType */ ;
```

The input type to a wrapping constructor. Typically this will be `value_type*`.

```
using size_type = /* implementation defined */ ;
```

The type used for extents.

```
using iterator = /* deduced from DataType and Properties */ ;  
using const_iterator = /* deduced from DataType and Properties */ ;
```

The type used to iterate elements of a contiguous array.

1.3.4 Constructors, copy, assignment, destructor

```
constexpr array_ref() noexcept
```

Effect: Construct a *null* `array_ref` with `data() == nullptr` and `extent(i) == 0` for all dynamic dimensions.

```
constexpr array_ref( const array_ref & rhs ) noexcept
```

Effect: Construct an `array_ref` of the same span of objects referenced by `rhs`.

Remark: There may be other `Properties...` dependent effects.

```
constexpr array_ref( array_ref && rhs ) noexcept
```

Effect: Construct an `array_ref` the span of objects referenced by `rhs` and then `rhs` is a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

```
template< typename UType , typename ... UProperties >  
constexpr array_ref( const array_ref< UType , UProperties ... > & rhs ) noexcept
```

Requires: `array_properties::is_assignable_v< array_ref , array_ref< UType , UProperties ... > >` Each static extent of `*this` is equal to the corresponding extent of `rhs`.

Effect: Construct an `array_ref` of the same span of objects referenced by `rhs`.

Remark: There may be other `Properties...` dependent effects.

```
template< typename UType , typename ... UProperties >  
array_ref & operator = ( const array_ref< UType , UProperties ... > & rhs ) noexcept
```

Requires: `array_properties::is_assignable_v< array_ref , array_ref< UType , UProperties ... > >`. Each static extent of `*this` is equal to the corresponding extent of `rhs`.

Effect: Construct an `array_ref` the span of objects referenced by `rhs` and then `rhs` is a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

```
array_ref & operator = ( const array_ref & rhs ) noexcept
```

Effect: Assigns `this` to reference the same span of objects referenced by `rhs`.

Remark: There may be other `Properties...` dependent effects.

```
array_ref & operator = ( array_ref && rhs ) noexcept = default
```

Effect: Assigns `this` to reference the same span of objects referenced by `rhs` and then `rhs` is a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

```
template< typename ... IntegralArgs >  
constexpr array_ref( pointer ptr , IntegralArgs ... dynamic_dims ) noexcept
```

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value`. Each `dynamic_dims` is non-negative. The span of objects denoted by `[ ptr , ptr + S )`, where `S = array_ref::required_span( dynamic_dims... )`, shall be a valid contiguous span of objects.

Remark: Shall not participate in overload resolution unless all `IntegralArgs` are integral types.

Effects: The *wrapping constructor* constructs a multidimensional array reference into the contiguous span of objects denoted by `[ ptr , ptr + S )` where `S = array_ref::span()` for this. The dimensions of the array are defined by replacing in order each dynamic extent in the `array_ref` type with a value from `dynamic_dims...`.

```
constexpr array_ref( pointer ptr , layout const& lay ) noexcept
```

Requires: The span of objects denoted by `[ ptr , ptr + S )`, where `S = array_ref::required_span( lay )`, shall be a valid contiguous span of objects.

Effects: The *wrapping constructor* constructs a multidimensional array reference of the given member memory such that all data members are in the span `[ ptr , ptr + lay.span() )`.

```
~array_ref()
```

Effect: Assigns this to be a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

1.3.5 Observers of the domain index space

Observers of the domain index space correspond to the observers of `array_property::dimension_typed`.

```
static constexpr size_type rank() noexcept
static constexpr size_type rank_dynamic() noexcept
constexpr size_type extent( size_type ith ) const noexcept
constexpr size_type size() const noexcept
```

1.3.6 Observers of the layout mapping

Observers of the layout mapping correspond to observers of `array_property::/*layout_concept*/::mapping`.

```
static constexpr bool is_always_unique
static constexpr bool is_always_contiguous
static constexpr bool is_always_regular
constexpr bool is_unique() const noexcept ;
constexpr bool is_contiguous() const noexcept ;
constexpr bool is_regular() const noexcept ;
constexpr size_type span() const noexcept ;
template< typename IntegralType >
constexpr size_type stride( IntegralType index ) const noexcept

template< typename ... IntegralArgs >
static constexpr size_type required_span( IntegralArgs ... dynamic_dims ) noexcept
```

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value`. Each `dynamic_dims` is non-negative.

Returns: Required length of contiguous span of objects for the *wrapping constructor*. The dimensions of the array are defined by replacing in order each dynamic extent in the `array_ref` type with a value from `dynamic_dims...`.

Remark: As if constructing the corresponding layout mapping with the `dynamic_dims...` and querying the `span()` of this object.

```
static constexpr size_type required_span( layout const & ) noexcept
```

Returns: Required length of contiguous span of objects for the *wrapping constructor*.

Remark: As if constructing the corresponding layout mapping with the `layout` and querying the `span()` of this object.

1.3.7 Element and data access

```
constexpr pointer data() const noexcept
```

Returns: Pointer to the member object with the minimum location in the span of the array.

```
template< typename ... IntegralArgs >
reference operator()( IntegralArgs ... indices ) const noexcept
```

Requires: `conjunction<is_integral<IntegralArgs>::type...>::value.rank() <= sizeof...(IntegralArgs)`. The `ith` coordinate of `indices...`, denoted as `indices[ith]`, is valid: `0 <= indices[ith] < extent(ith)`.

Returns: A reference to the member object referenced by `indices...`.

Remark: An implementation may have rank-specific overloads to better enable optimization of the member access operator. Since `extent(ith) == 1` for `rank() <= ith` then extra zero-value indices are valid.

```
template< typename IntegralType >
reference operator[]( IntegralType index ) const noexcept
```

Requires: `is_integral<IntegralType>::value.rank() == 1`, `0 <= i < extent(0)`.

Returns: A reference to the member object referenced by `index`.

Requires: `0 <= i < extent(0)`

```
constexpr iterator begin() const noexcept
constexpr iterator end() const noexcept
constexpr const_iterator cbegin() const noexcept
constexpr const_iterator cend() const noexcept
constexpr reverse_iterator rbegin() const noexcept
constexpr reverse_iterator rend() const noexcept
constexpr reverse_const_iterator crbegin() const noexcept
constexpr reverse_const_iterator crend() const noexcept
```

Requires: `is_contiguous()`

Returns: iterator for member objects `[data(), data()+span())`.

Remark: The order of iteration is unspecified and layout dependent. If the layout mapping **is** unique then iterating the span is equivalent to iterating all indices of the domain multidimensional index space.

1.4 array_property::is_assignable meta-function

```
template< class array_ref_V , class array_ref_U >
array_property::is_assignable< array_ref_V , array_ref_U >
```

Requires: `array_ref_V` is `array_ref< DataType_V , Properties_V... >`, `array_ref_U` is `array_ref< DataType_U , Properties_U... >`

Returns: Deduce the potential assignability of `array_ref` objects from their `DataType` and `Properties...`. A `true_type` result requires assignability of the value type, equal rank, and compatibility of extents. A dynamic extent may be assigned from a static extent or dynamic extent. A static extent may only be *assigned* from an equal value static extent. A static extent is potentially *assignable* from a dynamic extent, if the dynamic extent has equal value.

Indicates if objects potentially non-identical `array_ref` types are assignable.

Assignability is deduced from properties of the `array_ref`.

1.5 array_property::bounds_check

```
template< bool Enable > struct bounds_check_if ;
using bounds_check = bounds_check<true> ;
```

When `array_ref Properties...` includes `bounds_check_if<true>` then the mapping operators `array_ref::operator()` and `array_ref::operator[]` verify that each index is valid, `0 <= indices[ith] < extent(ith)`. Verification failure shall be reported.

1.6 subarrays

```
template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
array_ref< /* deduced */ >
subarray( array_ref< DataType, Properties ... > const & ar , SliceSpecifiers ... specs) noexcept
```

Requires: `ar.rank() == sizeof...(SliceSpecifiers)`. The *ith* member of the `SliceSpecifier...specs` argument pack is an integral *value* or an integral *range* denoted by one of the following.

- an `initializer_list<T>` of integral type `T` and size 2
- a `pair<T,T>` of integral type `T`
- a `tuple<T,T>` of integral type `T`
- an `array<T,2>` of integral type `T`
- `array_property::all` which denotes `[0..extent(ith))`

The *ith* member of `SliceSpecifiers...specs` must be within the range `[0,ar.extent(ith))`.

- If an integral value then $0 \leq \textit{value} < \text{ar.extent}(\textit{ith})$
- If an integral range then $0 \leq \textit{begin} \leq \textit{end} \leq \text{ar.extent}(\textit{ith})$

Returns: An `array_ref` referring into the same memory extent as `ar`, with dimensions, layout, and other properties deduced from `ar` and the `SlicerSpecifier...specs` argument pack. The returned `array_ref` `rank()` is one less than the rank of `ar` for each member of the argument pack that is an integral value. The returned `array_ref` `extent(ith)` is equal to **end-begin** of the *ith* integral range argument

Remark: Subarray type deduction should generate an `array_ref` with the best performing layout mapping. A generated subarray `array_ref` that is always given `layout_stride` will be less performant than a equivalent subarray with `layout_right` OR `layout_left`.